

Publication Number: CN 122594138 A
PUBLISHED ON 18-Aug-2026

Application Number: CN 202610640294 A
FILED ON 9-May-2026

自动测试方法、装置、存储介质、设备及程序产品 (Chinese)

Automatic test method, device, storage medium, apparatus and program product (English)

China Invention Application Publication (Source: IFI)

Inventors: YU QIULIN
(Guangzhou City, Tianhe District,
Guangdong Province, CN)
LU TINGTING

Classifications: International (2023.01): G06N 5/022
International (2025.01): G06F 11/3668

Applicants: GUANGZHOU BOGUAN
INFORMATION SCIENCE &
TECHNOLOGY Co.,Ltd.
(Guangzhou City, Tianhe District,
Guangdong Province, CN)

Attorney, Agent or Firm: Zheng Xiaohui

Abstract (Chinese)

本申请公开了一种自动测试方法、装置、存储介质、设备及程序产品，包括：基于知识图谱和/或经验库处理测试指令以生成测试操作序列；所述知识图谱基于执行测试操作的数据进行语义解析后得到的结果构建，所述知识图谱至少包括页面节点、元素节点以及节点之间的转换关系；所述经验库用于存储操作序列的语义向量及对应的操作步骤；执行所述测试操作序列，通过知识图谱与经验库共同支撑测试指令处理，实现测试逻辑的结构表达（知识图谱）与语义理解（经验库），克服依赖固定脚本的局限性、提高测试效率以满足测试需求，同时提升测试系统的语义理解能力与自适应决策能力。

[Image Omitted]

Abstract (English)

The application discloses an automatic test method, device, storage medium, equipment and program product, which comprises the steps of processing test instructions based on a knowledge graph and/or an experience library to generate a test operation sequence, constructing the knowledge graph based on a result obtained by carrying out semantic analysis on data of test operation, wherein the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, the experience library is used for storing semantic vectors of the operation sequence and corresponding operation steps, executing the test operation sequence, and supporting test instruction processing through the knowledge graph and the experience library together to realize structural expression (knowledge graph) and semantic understanding (experience library) of test logic, overcome the limitation of relying on fixed scripts, improve test efficiency to meet test requirements, and improve the semantic understanding capability and self-adaptive decision capability of a test system.

[Image Omitted]

Synopsis AI Document Summarization (Default)

Summary Size: Default

Key Phrases

- test operation
- test operation sequence
- processing test instructions based
- knowledge graph

The embodiment of the application provides an automatic test method, an automatic test device, a storage medium, equipment and a program product, which can solve at least one technical problem.

In one aspect, an embodiment of the present application provides an automatic test method, including:

the method comprises the steps of processing a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and at least comprises page nodes, element nodes and conversion relations among the nodes;

And executing the test operation sequence.

In another aspect, an embodiment of the present application provides an automatic test apparatus, including:

The system comprises a generation module, a test operation sequence generation module and a test operation sequence generation module, wherein the generation module is used for processing a test instruction based on a knowledge graph and/or an experience library, the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes;

and the execution module is used for executing the test operation sequence.

The automatic test method provided by the embodiment of the application is used for generating a test operation sequence by processing test instructions based on a knowledge graph and/or an experience library, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of executing test operations, the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, the experience library is used for storing semantic vectors of the operation sequence and corresponding operation steps, and the test operation sequence is executed, so that the structure expression (knowledge graph) and semantic understanding (experience library) of test logic are realized, the limitation of relying on fixed scripts is overcome, the test efficiency is improved to meet test requirements, and the semantic understanding capability and the self-adaptive decision capability of a test system are improved.

Referring to fig. 2, fig. 2 is a flow chart of an automatic test method according to an embodiment of the application. It should be noted that the steps shown may be performed in a different logical order than that shown in the method flow diagram. The method provides a graphical user interface through the terminal device, the graphical user interface comprising at least a portion of the virtual scene and at least one virtual character, the method may comprise the steps of:

Step 011, processing a test instruction based on a knowledge graph (database) and/or an experience database (vector library) to generate a test operation sequence, constructing the knowledge graph based on a result obtained by performing semantic analysis on data of the test operation, wherein the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, and the experience database is used for storing semantic vectors of the operation sequence and corresponding operation steps.

The knowledge graph can be a GUI semantic network constructed based on GUI test operation data after multi-mode semantic analysis, and a graph database (such as Neo4j) is used as a storage carrier and can be used for representing logical association of pages, elements and operations. For convenience of explanation, the embodiment of the application takes a Neo4j database as an example of a knowledge graph.

Specifically, the multi-modal large model can be trained (the training can be online training or offline training) by adopting multi-modal operation track data comprising first page data, second page data and test operation action data, so that the model can master the semantic analysis and logical reasoning capability of the GUI test. And then analyzing the multi-mode operation track data by calling the trained multi-mode large model to extract page types, page element semantics, operation causal relationships and the like before and after the test operation so as to form a standardized semantic analysis result. And finally, inputting a natural language test instruction into the multi-mode large model, so that the multi-mode large model is combined with page-element-conversion logic of the knowledge graph and historical operation sequence semantic information of the experience library to infer and generate a test operation sequence meeting test requirements.

Step 012, execute test operation sequence.

Specifically, the generated test operation sequence can be used as an execution blue book, real interaction behavior of a human user is simulated through the GUI automatic execution engine, operations such as clicking, inputting, sliding and the like are executed on target elements one by one according to a preset sequence, and the business process corresponding to the test instruction is completed. For example, when the server is connected to the tested terminal (the tested terminal is provided with the tested application program), the server may issue the corresponding operation instructions to the tested terminal one by one according to the step sequence of the test operation sequence, and the tested terminal sequentially executes the corresponding GUI interaction actions according to the operation instructions, so as to implement execution of the test operation sequence.

The method comprises the steps of generating a test operation sequence by processing test instructions based on a knowledge graph and/or an experience library, constructing the result obtained by carrying out semantic analysis on the knowledge graph based on data for executing the test operation, wherein the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, the experience library is used for storing semantic vectors of the operation sequence and corresponding operation steps, executing the test operation sequence, and supporting the test instruction processing together with the experience library by the knowledge graph, so that structural expression (knowledge graph) and semantic understanding (experience library) of test logic are realized, the limitation of relying on fixed scripts is overcome, and the semantic understanding capability and self-adaptive decision capability of a test system are improved.

In some embodiments, step 011, processing test instructions based on knowledge maps and/or experience libraries to generate a sequence of test operations, includes:

Step 0112, according to the test content and the test step of the test instruction, carrying out complexity evaluation on the test instruction to obtain a complexity parameter;

0113, under the condition that the complexity parameter is smaller than a first preset complexity threshold value, selecting a target operation sequence matched with the test instruction from the experience library, and modifying the target operation sequence based on the test instruction and the experience map to generate a test operation sequence;

Step 01121, performing similarity calculation on the test semantic vector and the semantic vector of each operation sequence stored in the experience library to obtain each similarity score, determining the similarity score with the largest numerical value as the similarity score, and performing semantic vectorization on the test semantic vector based on the test instruction to obtain the test semantic vector;

Step 01122, carrying out semantic analysis on the test instruction to obtain a test semantic analysis result, carrying out matching calculation on the test semantic analysis result and the knowledge graph, and determining a matching degree score according to the calculation result;

The semantic vector similarity calculation can be to calculate the matching degree of the test semantic vector and the operation sequence semantic vector in the experience library by adopting cosine similarity and other algorithms, and output the corresponding similarity score.

Optionally, step 0113, modifying the target sequence of operations based on the test instructions and the empirical spectrum to generate a test sequence of operations, includes:

step 01132, carrying out semantic analysis on the test instruction to obtain a test page node, a test element node and a test conversion relation among the nodes corresponding to the test instruction;

Dividing the test page node, the test element node and the test conversion relation among the nodes corresponding to the test instruction according to the experience map to obtain effective parameters and failure parameters, wherein the effective parameters comprise the

effective test page node, the effective test element node and the effective test conversion relation matched with the test conversion relation among the test page node, the test element node and the nodes corresponding to the test instruction in the experience map, and the failure parameters comprise the effective test page node, the effective test element node and the effective test conversion relation which are not matched with the test conversion relation among the test page node, the test element node and the nodes corresponding to the test instruction in the experience map;

Step 01134, selecting a replacement parameter matched with the failure parameter from the empirical spectrum according to a preset similarity condition;

And 01135, modifying the target operation sequence based on the effective parameter and the replacement parameter to generate a test operation sequence.

In order to facilitate better implementation of the automatic test method of the embodiment of the application, the embodiment of the application also provides an automatic test device. Referring to the drawings, the structure of an automatic test device according to an embodiment of the application is shown. The automatic test equipment 200 may include:

the generation module 201 is configured to process a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, where the knowledge graph is constructed based on a result obtained by performing semantic analysis on data of a test operation, and the knowledge graph at least includes page nodes, element nodes and conversion relations between the nodes;

an execution module 202, configured to execute the test operation sequence.

And executing the test operation sequence.

Document Description

Automatic test method, device, storage medium, apparatus and program product

Technical Field

The present application relates to the field of testing technologies, and in particular, to an automatic testing method, apparatus, storage medium, device, and program product.

Background

In the field of software testing, automated testing of Graphical User Interfaces (GUIs) is a key element in ensuring the quality of applications. Related art generally relies on pre-written test scripts to perform tests by simulating user operations (e.g., clicks, inputs). These scripts often perform element positioning and manipulation based on a fixed interface element identifier (e.g., ID, XPath) or screen coordinates.

However, as application iteration speeds increase and cross-platform testing requirements increase, frequent reform of UI interfaces results. Once the position, attribute or overall layout of the interface element changes, the original script cannot accurately position the element, so that test execution fails, a large amount of time is required for a tester to perform manual investigation and updating of the script, the test efficiency is low, and the test requirement is difficult to adapt.

Disclosure of Invention

The embodiment of the application provides an automatic test method, an automatic test device, a storage medium, equipment and a program product, which can solve at least one technical problem.

In one aspect, an embodiment of the present application provides an automatic test method, including:

the method comprises the steps of processing a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and at least comprises page nodes, element nodes and conversion relations among the nodes;

And executing the test operation sequence.

In another aspect, an embodiment of the present application provides an automatic test apparatus, including:

The system comprises a generation module, a test operation sequence generation module and a test operation sequence generation module, wherein the generation module is used for processing a test instruction based on a knowledge graph and/or an experience library, the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes;

and the execution module is used for executing the test operation sequence.

In another aspect, embodiments of the present application provide a computer readable storage medium storing a computer program adapted to be loaded by a processor to perform the automatic test method of any of the embodiments described above.

In another aspect, an embodiment of the present application provides a computer device, where the computer device includes a processor and a memory, where the memory stores a computer program, and the processor is configured to execute the automatic test method according to any one of the embodiments above by calling the computer program stored in the memory.

In another aspect, embodiments of the present application provide a computer program product comprising computer instructions which, when executed by a processor, implement the automatic test method as described in any of the embodiments above.

The automatic test method provided by the embodiment of the application is used for generating a test operation sequence by processing test instructions based on a knowledge graph and/or an experience library, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of executing test operations, the knowledge graph at least comprises page nodes,

element nodes and conversion relations among the nodes, the experience library is used for storing semantic vectors of the operation sequence and corresponding operation steps, and the test operation sequence is executed, so that the structure expression (knowledge graph) and semantic understanding (experience library) of test logic are realized, the limitation of relying on fixed scripts is overcome, the test efficiency is improved to meet test requirements, and the semantic understanding capability and the self-adaptive decision capability of a test system are improved.

Drawings

In order to more clearly illustrate the technical solutions of the embodiments of the present application, the drawings that are needed in the description of the embodiments will be briefly described below, it being obvious that the drawings in the following description are only some embodiments of the present application, and that other drawings may be obtained according to these drawings without inventive effort for a person skilled in the art.

Fig. 1 is a schematic diagram of an exemplary test system according to an embodiment of the present application.

Fig. 2 is a flow chart of an automatic test method according to an embodiment of the present application.

Fig. 3 is a schematic diagram of a first application scenario of an automatic test method according to an embodiment of the present application.

Fig. 4 is a schematic diagram of a second application scenario of the automatic test method according to the embodiment of the present application.

Fig. 5 is a schematic diagram of a third application scenario of the automatic test method according to the embodiment of the present application.

Fig. 6 is a schematic structural diagram of an automatic test device according to an embodiment of the present application.

Fig. 7 is a schematic structural diagram of a computer device according to an embodiment of the present application.

Detailed Description

The following description of the embodiments of the present application will be made clearly and completely with reference to the accompanying drawings, in which it is apparent that the embodiments described are only some embodiments of the present application, but not all embodiments. All other embodiments, which can be made by those skilled in the art based on the embodiments of the application without making any inventive effort, are intended to fall within the scope of the application.

The background of the application is described further below.

Graphical user interface (GRAPHICAL USER INTERFACE, GUI) automated testing is an important branch of the software testing arts for verifying that the graphical interface of an application is working as intended. Conventional GUI automation testing relies on pre-written test scripts to perform test tasks by simulating user operations (e.g., clicking, entering, sliding, etc.).

In the related art, GUI automation test frameworks (e.g., selenium, appium, UIAutomator, etc.) position and manipulate interface elements primarily by (1) coordinate-based positioning by clicking on the element with fixed screen coordinates, (2) element ID/XPath-based positioning by relying on a developer-predefined identifier, and (3) image recognition-based positioning by template matching or OCR recognition of the element.

However, as application iteration speed increases, UI changes frequently, and cross-platform testing requirements increase, traditional GUI automated testing faces challenges of high maintenance cost, strong vulnerability, low level of intelligence, and the like.

For example, taking a takeaway program running on a tested terminal as an example, in the process that a user logs in to the takeaway program and interacts with a control on the takeaway program to realize a takeaway order, an application program needs to display different pages and execute different operation logics in response to the operation of the user. GUI testing is the process and results of testing the presentation and response of multiple pages from a user login account, on-program options, ordering, etc.

For example, in a script-based recording and playback technique, a tester manually operates an application program through a recording tool, and the system automatically records operation steps and generates a test script, and when recording and playback, tests are executed according to element positioning information and operation sequences recorded in the script. The specific process can be embodied in that in the recording stage, the script is regenerated by capturing user operation first and then recording the XPath/ID of the element, in the playback stage, the script is read first, then the element is positioned, and finally the operation is executed and the result is verified.

However, the solution does not understand the business semantics and user operation intent of the interface, which results in a large amount of underlying element location code being written by the tester, and the test requirements being unable to be described in natural language, and all relevant test scripts still being manually modified when the interface UI is modified but the functional logic is unchanged (e.g., the "buy" button moves from top to bottom of the page), and the solution also does not handle synonymous operations (e.g., in the take-away program example, "join shopping cart" is interchangeable with "buy immediately" in some scenarios).

For example, in one computer vision based element recognition technique, elements on a screen are recognized using template matching or OCR techniques. The system stores the image templates of the elements in advance, and the elements are matched and positioned through the image similarity when the test is executed. The implementation process can comprise the steps of saving a screenshot of a target element as a template in a preparation stage, carrying out image matching according to the screenshot in an execution stage and carrying out test operation according to image coordinates, and further constructing a positioning rule by combining various attributes (text, position, father node and the like) of the element in a rule-based intelligent positioning technology, and automatically trying other attributes when one attribute fails. A typical implementation flow includes setting multiple alternate locators (ID, XPath, text) for elements during a definition phase, and attempting locators by priority during an execution phase and performing operations for the first successful locator.

However, when performing complex multi-step test tasks, these schemes must explicitly plan the precise sequence of each step operation in advance, lack the automatic planning capability of target guidance, and cannot handle dynamic page flows (such as a/B test, region differentiated interface, conditional popup), and cannot intelligently select an optimal path when multiple feasible paths exist for realizing the same test target.

Furthermore, it is counted that after each application version update, a significant proportion of UI automation test cases will be interrupted due to element positioning failure. The existing framework can only report error information (such as Element not found), cannot automatically analyze failure reasons and attempt repair, and needs to manually check failed test cases one by one.

Finally, although a large number of screen shots are captured in the related art, the related art is only used for manual examination, and semantic features such as layout, color, icons and the like are not extracted by using computer vision technology. Cross-modal semantic associations between "visual features-element functions-operational intents" are not established.

In view of the above, embodiments of the present application provide an automatic test method, an apparatus, a storage medium, a device, and a program product. Specifically, the automatic test method of the embodiment of the application can be executed by computer equipment, wherein the computer equipment can be equipment such as a tested terminal or a server. The tested terminal can be a smart phone, a tablet personal computer, a notebook computer, an intelligent television, a wearable intelligent device, an intelligent vehicle-mounted terminal and other devices, and the tested terminal can also comprise a client, wherein the client can be a client, a browser client, an instant messaging client or an applet and the like which are installed on the tested device. The server may be an independent physical server, a server cluster or a distributed system formed by a plurality of physical servers, or a cloud server providing cloud services, cloud databases, cloud computing, cloud functions, cloud storage, network services, cloud communication, middleware services, domain name services, security services, a content distribution network (Content Delivery Network, CDN), basic cloud computing services such as big data and an artificial intelligent platform.

For example, when the automatic test method is run on a terminal device, the terminal device may include a display screen for presenting a graphical interface of an application to be tested and receiving instructions generated by a user acting on the graphical interface, and a processor. The graphical interface may include an operator interface, a display interface, an operator control, and the like. The processor is used for storing an application program to be tested, running the application program, generating a graphical interface, responding to the instruction and controlling the display of the graphical interface on the display screen. When the user operates the graphical interface through the display screen, the graphical interface can control the local content of the terminal equipment by responding to the received operation instruction. The way in which the terminal device presents the graphical user interface to the user may include a variety of ways, for example, the graphical user interface may be rendered for display on a display screen of the terminal device, or presented by holographic projection.

For example, when the automatic test method is run on a server, the method may be implemented and executed based on a cloud test system. Cloud testing systems refer to application program modes based on cloud computing. The cloud testing system comprises a server and client equipment. The running main body of the application program and the graphic interface presentation main body are separated, and the storage and the running of the automatic test method are completed on a server. The graphical interface presentation is performed at a client, where the client is mainly used for receiving and sending application data and presenting the graphical interface, for example, the client may be a display device with a data transmission function near a user side, such as a mobile terminal, a television, a computer, a palm computer, a personal digital assistant, a head-mounted display device, etc., but the terminal device performing application data processing is a cloud server. When the application program is carried out, a user operates the client to send an instruction to the server, the server controls the application program to run according to the instruction, data such as a graphical interface and the like are encoded and compressed, the data is returned to the client through a network, and finally, the data is decoded through the client and the graphical interface is output.

It should be noted that, in the embodiment of the present application, the execution body of the automatic test method may be a terminal device or a server, where the terminal device may be a local terminal device or a client device in the aforementioned cloud application. The embodiment of the application does not limit the type of the execution body.

It will be appreciated that in the specific embodiment of the present application, related data such as object data, context data, etc. of a user are involved, when the embodiments of the present application are applied to specific products or technologies, user permission or consent is required, and the collection, use, and processing of related data is required to comply with relevant laws and regulations and standards of relevant countries and regions.

By way of example, in connection with the above description, fig. 1 shows a test system 1000 for implementing an automatic test method according to an embodiment of the present application, where the test system 1000 may include at least one terminal 1001, at least one server 1002, at least one database 1003, and a network. The terminal 1001 held by the user may be connected to different servers through a network. A terminal is any device having computing hardware capable of supporting software application tools corresponding to executing applications.

The test system 1000 employs a hierarchical server-client architecture that can execute a sequence of test operations at a client by generating the sequence of test operations at a server.

In the test system 1000 described above, the terminal 1001 is used to install and run an application, and in some cases, the application may not be installed in advance in the terminal 1001, and a user may directly access the incoming application through a client such as a browser. The user logs in the application program by using the registered application program account number, and can participate in the application of the application program. When a user logs in an application, the terminal 1001 sends a login request to the server 1002, the server 1002 verifies an account number of the application used by the user, and the server 1002 determines an application mechanism corresponding to the application account number according to the login request, and the verification returns a login success notification to the terminal 1001. In the process of applying an application program by a user, data interaction is performed between the terminal 1001 and the server 1002, the terminal 1001 transmits various information to the server 1002, the server 1002 determines display data of the terminal 1001 according to a stored application program mechanism and the received information, and transmits the display data to the terminal 1001, so that the display data transmitted by the server 1002 is presented to the user through the terminal 1001.

In a possible application scenario, different terminals 1001 may be served by different servers 1002, so in order to distinguish between the servers 1002 corresponding to the different application terminals 1001, the first and second modes will be described when they are referred to in the embodiment of the present application. In practice, the servers 1002 corresponding to the different application terminals 1001 may be the same server 1002. It can be understood that the terminals 1001 corresponding to virtual characters located in the same application scene are served by the same server 1002 without distinguishing the first and second.

In addition, when the test system 1000 includes a plurality of terminals, a plurality of servers, and a plurality of networks, different terminals may be connected to each other through different networks, through different servers. The network may be a wireless network or a wired network, such as a Wireless Local Area Network (WLAN), a Local Area Network (LAN), a cellular network, a 2G network, a 3G network, a 4G network, a 5G network, etc. In addition, the different terminals may be connected to other terminals or to a server or the like using their own bluetooth network or hotspot network. In addition, the system 100 may include multiple databases coupled to different servers and information related to the application may be continuously stored in the databases as different users conduct multi-user applications online.

It should be noted that, in the embodiment of the present application, the same virtual application is running on a plurality of terminal devices, so that data interaction between a plurality of terminal devices may be implemented through a server of the virtual application.

Thus, the terminal device 1 sending data to the terminal device 2 may be understood as the terminal device 1 sending data to the server of the virtual application, which server sends data to the terminal device 2. The reception of data transmitted by the terminal device 2 by the terminal device 1 is understood to be the reception of data transmitted by the server of the virtual application by the terminal device 1, which is the data transmitted by the terminal device 2 to the server. Or there may be no application server, and the terminal device 1 directly transmits the application data to the terminal device 2.

It should be noted that the schematic diagram of the test system shown in fig. 1 is only an example, and the test system 1000 described in the embodiment of the present application is for more clearly describing the technical solution of the embodiment of the present application, and does not constitute a limitation to the technical solution provided in the embodiment of the present application, and those skilled in the art can know that the technical solution provided in the embodiment of the present application is equally applicable to similar technical problems with evolution of the test system and occurrence of new service scenarios.

It should be noted that, the triggering operations that occur in the following detailed description of the automatic test method provided by the embodiment of the present application may be regarded as triggering operations implemented by a user through a finger or a medium such as a control mouse, a keyboard, or a stylus. Which medium is specifically used may be determined according to the type of computer device. For example, when the computer device is a touch screen device such as a cell phone, tablet, application, etc., the user may operate on the touch screen via any suitable object or accessory such as a finger, stylus, etc. When the terminal equipment is a non-touch screen terminal equipment such as a desktop computer, a notebook computer and the like, a user can operate the terminal equipment through external equipment such as a mouse, a keyboard and the like.

The technical scheme of the application is described in detail through specific examples. It should be noted that the following embodiments may be combined with each other, and the same or similar concepts or processes may not be described in detail in some embodiments.

Referring to fig. 2, fig. 2 is a flow chart of an automatic test method according to an embodiment of the application. It should be noted that the steps shown may be performed in a different logical order than that shown in the method flow diagram. The method provides a graphical user interface through the terminal device, the graphical user interface comprising at least a portion of the virtual scene and at least one virtual character, the method may comprise the steps of:

Step 011, processing a test instruction based on a knowledge graph (database) and/or an experience database (vector library) to generate a test operation sequence, constructing the knowledge graph based on a result obtained by performing semantic analysis on data of the test operation, wherein the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, and the experience database is used for storing semantic vectors of the operation sequence and corresponding operation steps.

The knowledge graph can be a GUI semantic network constructed based on GUI test operation data after multi-mode semantic analysis, and a graph database (such as Neo4j) is used as a storage carrier and can be used for representing logical association of pages, elements and operations. For convenience of explanation, the embodiment of the application takes a Neo4j database as an example of a knowledge graph.

The experience library can take a vector database (such as Milvus) as a storage carrier, is used for storing semantic vectors of the operation sequences and databases corresponding to executable operation steps, and can realize quick multiplexing of historical test experiences through vector similarity retrieval.

The test instruction may be a GUI test requirement (e.g., "search for merchandise and view details") entered in natural language.

The test operation sequence may be an operation set that may be directly executed on the GUI of the tested terminal, and may include specific operations such as clicking, inputting, sliding, and the execution sequence.

The semantic vector can be a high-dimensional value vector which is used for realizing similarity matching at a semantic level by converting business semantics of an operation sequence into a large language model, and is not based on physical feature matching of coordinates and IDs.

The page nodes can be basic nodes (such as a top page, an item detail page, a login page and the like) used for representing the GUI page type, layout and screenshot information in the knowledge graph.

The element nodes can be nodes (such as search boxes, purchase buttons, return keys, and the like) in the knowledge graph, which are used for representing semantic features, visual features, and position information of the interactable elements in the GUI.

The conversion relation can be an edge representing the skip logic among pages in the knowledge graph, and the operation type, the target element and the business intention of triggering skip are recorded.

Optionally, the multi-mode operation track data (for example, including a screen shot before operation, a screen shot after operation, a specific operation interaction action and the like) is obtained through manual demonstration and/or video recording demonstration and/or custom exploration, after semantic analysis is performed on the multi-mode operation track data, structural knowledge of pages, elements and operation conversion is constructed, a knowledge graph can be obtained, and the knowledge graph can realize logic rationality verification and unknown scene reasoning of a test path.

Optionally, the test operation sequence successfully executed by the history is converted into a semantic vector to be stored, so as to generate an experience library (vector library), and through vector similarity matching, experience quick multiplexing of similar test tasks can be realized.

Optionally, step 011, processing the test instructions based on knowledge-graph and/or experience-base to generate a sequence of test operations, including:

And 0111, calling a multi-mode large model, and carrying out semantic understanding on the test instruction by combining a knowledge graph and/or an experience library so as to generate a test operation sequence.

The multi-mode large model is trained by multi-mode operation track data, and the multi-mode operation track data at least comprises first page data before test operation is executed, second page data after the test operation is executed and test operation action data.

The multi-mode large model performs semantic analysis on the multi-mode operation track data to obtain semantic analysis results, wherein the semantic analysis results comprise page types before and after test operation execution, element semantics of the page and operation causal relationships, and operation sequences obtained through analysis are subjected to semantic vectorization and stored in an experience library.

The multi-modal large model can be a large model integrating visual understanding capability and natural language understanding capability and is used for carrying out cross-modal semantic analysis and processing on images, texts and operation behaviors of the GUI test. Can include the common construction of a visual Language multimodal Model (VLM) and a large Language Model (Large Language Model, LLM)

The multi-mode operation track data can be multi-dimensional data for recording the complete interaction process of the GUI test, can provide a data source for model training and semantic analysis, and at least comprises at least one of first page data before test operation execution, second page data after test operation execution and test operation action data.

The first page data may be GUI page raw data before execution of the test operation, including visualization and structural information such as a screenshot, a tree of page controls, and a layout of a page. For example, the first page data may be a pre-operation screenshot S_1 , in PNG format, with a device-specific resolution (e.g., 1080X 1920 pixels) stored in client memory.

The second page data may be GUI page raw data after the test operation is performed, and is compared with the first page data to form a front-back state, so as to analyze page changes caused by the operation. For example, the second page data may be a post-operation screenshot S_2 , in PNG format, with a device-specific resolution (e.g., 1080X 1920 pixels) stored in client memory.

The test operation action data may be interactive operation information executed on the GUI, and include executable action data such as operation type (e.g., clicking, inputting, sliding, etc.), operation target, operation parameter, etc. The test operation action data may be JSON format records containing action type (click/input/wipe), target element coordinates, input text, etc.

The semantic analysis result may be structural semantic information output after the multi-mode large model analyzes the multi-mode operation track data.

The page type may be a service classification (such as a home page, a search page, an item detail page, a login page, etc.) of the GUI page, and represents a functional attribute of the page.

The element semantics can be business meaning and function positioning (such as search box and immediate purchase button) of the GUI interactable element.

The operation causal relationship may be a logical association between a test operation and a page status change, that is, a result of a page jump, a content update, a status change, etc. caused by executing a certain operation.

The semantic vectorization can be a process of converting the business semantics of the operation sequence into a high-dimension value vector, so that the semantic representation and the quick similarity retrieval of the operation sequence are realized.

The model has the special capabilities of GUI page understanding, element semantic recognition and operation logic reasoning, and is adapted to a GUI automation test scene. The multi-mode operation track data is used as input, the data barriers for breaking images, texts and operation behaviors are realized based on the multi-mode large model, deep semantic association of pages, elements and operations is extracted, and unstructured interaction data is converted into the effect of structured semantic information. And constructing a knowledge graph through a semantic analysis result, storing the operation sequence semantic vectorization into an experience library, and calling a multi-mode large model to combine the logic reasoning capacity and the experience multiplexing capacity of the double libraries, so as to generate a test operation sequence conforming to a test instruction.

Specifically, the multi-modal large model can be trained (the training can be online training or offline training) by adopting multi-modal operation track data comprising first page data, second page data and test operation action data, so that the model can master the semantic analysis and logical reasoning capability of the GUI test. And then analyzing the multi-mode operation track data by calling the trained multi-mode large model to extract page types, page element semantics, operation causal relationships and the like before and after the test operation so as to form a standardized semantic analysis result. And finally, inputting a natural language test instruction into the multi-mode large model, so that the multi-mode large model is combined with page-element-conversion logic of the knowledge graph and historical operation sequence semantic information of the experience library to infer and generate a test operation sequence meeting test requirements.

That is, the pre-operation screenshot, the test operation and the post-operation screenshot of the GUI test can be collected to perform semantic analysis through a multi-mode large model (built together based on VLM and LLM), extract page types, element semantics and operation causal relationships, construct a knowledge graph of the GUI semantics comprising page nodes, element nodes and conversion relationships among the nodes, and store the knowledge graph in a graph database. Meanwhile, the test operation sequence successfully executed by the history is converted into an operation sequence semantic vector through a large language model, and the semantic vector and the corresponding operation steps are associated and stored in a vector database to form an experience library. Under the condition of receiving a test instruction of natural language, the test instruction can be converted into a semantic vector by relying on an experience library, the semantic vector with highest similarity is searched in the experience library, and corresponding operation steps are directly called to generate an operation sequence, or the operation sequence is generated by relying on a knowledge graph and based on the page-element-conversion relation of the knowledge graph, the execution path of the test instruction is reasoned and satisfied to generate the operation sequence, or the operation sequence is generated by combining the search result of the experience library and the logic verification of the knowledge graph, so that the test operation sequence is obtained. In practical application, the embodiment of the application can lead a tester to write simple natural language instructions, reduce the code quantity by tens of times or hundreds of times compared with the traditional mode, shorten the time for writing the test cases with medium complexity from a few hours to a ten-minute level, obviously improve the writing efficiency, generally avoid the problem of rewriting a large number of scripts in the traditional method because the natural language instructions are not required to be modified or only required to be adjusted slightly when the UI is changed, and reduce the maintenance cost in response.

It can be understood that by introducing a multi-mode large model to train and analyze track data containing page front and back states and operation actions, semantic information in the multi-mode operation track data, such as coordinate positions, is fully learned, page types, element semantics, operation causal relationships and the like are further extracted, semantic understanding of a GUI interface is realized, a knowledge graph is constructed through a model analysis result, and meanwhile, an operation sequence is vectorized and stored in an experience library, so that the generated test operation sequence can be ensured to conform to actual business logic and historical experience, and reliability of a test result is improved.

3-5, The multi-mode large model may include a client execution layer, an application service layer, an AI reasoning layer and a knowledge storage layer, where the client execution layer may be running on a tested device and is responsible for actual GUI operation and data collection and must be in the same running environment as the tested application, the application service layer may be deployed at a server end and is responsible for task scheduling and policy decision, and may be laterally expanded to support multi-device concurrent testing, the AI reasoning layer may be an independent reasoning service cluster and may dynamically adjust computing resources (GPU/CPU) according to load to avoid affecting service logic, and the knowledge storage layer may be persistent to store testing knowledge and support cross-project and cross-application experience multiplexing.

Step 012, execute test operation sequence.

Specifically, the generated test operation sequence can be used as an execution blue book, real interaction behavior of a human user is simulated through the GUI automatic execution engine, operations such as clicking, inputting, sliding and the like are executed on target elements one by one according to a preset sequence, and the business process corresponding to the test instruction is completed. For example, when the server is connected to the tested terminal (the tested terminal is provided with the tested application program), the server may issue the corresponding operation instructions to the tested terminal one by one according to the step sequence of the test operation sequence, and the tested terminal sequentially executes the corresponding GUI interaction actions according to the operation instructions, so as to implement execution of the test operation sequence.

The method comprises the steps of generating a test operation sequence by processing test instructions based on a knowledge graph and/or an experience library, constructing the result obtained by carrying out semantic analysis on the knowledge graph based on data for executing the test operation, wherein the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes, the experience library is used for storing semantic vectors of the operation sequence and corresponding operation steps, executing the test operation sequence, and supporting the test instruction processing together with the experience library by the knowledge graph, so that structural expression (knowledge graph) and semantic understanding (experience library) of test logic are realized, the limitation of relying on fixed scripts is overcome, and the semantic understanding capability and self-adaptive decision capability of a test system are improved.

In some embodiments, step 011, processing test instructions based on knowledge maps and/or experience libraries to generate a sequence of test operations, includes:

Step 0112, according to the test content and the test step of the test instruction, carrying out complexity evaluation on the test instruction to obtain a complexity parameter;

0113, under the condition that the complexity parameter is smaller than a first preset complexity threshold value, selecting a target operation sequence matched with the test instruction from the experience library, and modifying the target operation sequence based on the test instruction and the experience map to generate a test operation sequence;

And 0114, under the condition that the complexity parameter is larger than a first preset complexity threshold value, traversing at least one execution path of each test sub-instruction based on the knowledge graph to generate a candidate path set, and selecting a target path between adjacent test sub-instructions based on the candidate path set to generate a test operation sequence, wherein the test sub-instructions are obtained after the test sub-instructions are disassembled based on the test instructions.

The complexity evaluation may be a process of quantitatively calculating the execution difficulty of the test task according to the semantic features, the history matching degree, the path difficulty, the page strangeness degree and other dimensions of the test instruction.

The complexity parameter may be a numerical indicator for quantitatively characterizing the complexity of the test task.

The first preset complex threshold may be a preset critical value, which is used for dividing a specific generation process of the test operation sequence. For example, the first preset complex threshold may be 0.5, 0.6, 0.7, 0.8, 0.85, 0.9 or the like. For example, the generation process of the test operation sequence is divided into an empirical mode (the complexity parameter is smaller than the first preset complexity threshold) and an inference mode (the complexity parameter is larger than the first preset complexity threshold) according to the comparison of the complexity parameter and the first preset complexity threshold.

The target operation sequence can be a historical successful execution operation sequence with highest semantic matching degree with the current test instruction in the experience library.

The test sub-instruction may be a subdivided and independently executable minimum test task unit obtained by disassembling the whole test instruction according to service logic.

The execution path may be a GUI operation flow that is inferred based on a knowledge graph and completes a single test sub-instruction.

The candidate path set may be a summarized set of all possible execution paths corresponding to each test sub-instruction.

The target path may be an execution path between adjacent test sub-instructions which is preferably selected from the candidate path set, has optimal connectivity and is most stable to execute.

Optionally, step 0112, performing complexity evaluation on the test instruction according to the test content and the test step of the test instruction to obtain a complexity parameter, includes:

Step 01121, performing similarity calculation on the test semantic vector and the semantic vector of each operation sequence stored in the experience library to obtain each similarity score, determining the similarity score with the largest numerical value as the similarity score, and performing semantic vectorization on the test semantic vector based on the test instruction to obtain the test semantic vector;

Step 01122, carrying out semantic analysis on the test instruction to obtain a test semantic analysis result, carrying out matching calculation on the test semantic analysis result and the knowledge graph, and determining a matching degree score according to the calculation result;

Step 01123, calculating a predicted operation sequence for executing the test instruction based on the knowledge graph, and determining step scores according to the number of steps contained in the predicted operation sequence;

Step 01124, calculating the number of target pages appearing in the estimated operation sequence, and determining page scores, wherein the target pages comprise unrecorded pages in the knowledge graph;

And 01125, calculating weights of the similarity score, the matching score, the step score and the page score to determine complexity parameters.

The test semantic vector may be a high-dimensional value vector obtained by performing semantic vectorization processing on the test instruction, and is used for representing service semantics of the test instruction.

The semantic vector similarity calculation can be to calculate the matching degree of the test semantic vector and the operation sequence semantic vector in the experience library by adopting cosine similarity and other algorithms, and output the corresponding similarity score.

The similarity score may be a maximum value selected from all similarity scores, and represents a matching degree between a current test task and a historical success experience.

The test semantic analysis result may be structural semantic information such as a page type, element semantics, operation intention and the like obtained after the test instruction is subjected to semantic analysis.

The matching degree score can be a score obtained by matching and calculating the test semantic analysis result and the knowledge graph, and represents the matching degree of the test task and the existing GUI knowledge system.

The estimated operation sequence may be an expected operation flow which is obtained based on knowledge graph reasoning and is required for completing the test instruction.

The step score may be a score determined according to the number of steps of the estimated operation sequence, and the more the number of steps is, the higher the step score is, and the higher the complexity of task execution is represented.

The target page can be a non-recorded and non-stored GUI strange page in the specific knowledge graph, and is a quantitative index for evaluating unknown risks of the task.

The page score may be a score determined according to the number of unfamiliar target pages in the estimated operation sequence, where the more unfamiliar pages, the higher the page score.

The weight calculation may be to perform a weighted operation on the similarity score, the matching score, the step score, and the page score according to a preset weight coefficient, so as to obtain a final quantization result.

The method comprises the steps of matching historical experience, matching a knowledge system, executing a flow length and unknown page risk, respectively extracting quantifiable scores, covering difficulty of a test task, fusing multi-dimensional scores through weighted calculation, converting abstract and subjective complexity of the test task into unique, quantifiable and comparable complexity parameters, realizing the transition from subjective judgment to numerical, standardized and automatic complexity assessment, and guaranteeing scientificity and consistency of test strategy decision.

For example, the method comprises the steps of converting a test instruction into a test semantic vector, calculating the test semantic vector one by one through a vector similarity algorithm and semantic vectors of all operation sequences in an experience library to obtain all similarity scores, selecting a maximum value as a similarity score, extracting test semantic analysis results such as page types, element semantics and operation intentions through semantic analysis on the test instruction, matching the results with page nodes, element nodes and conversion relations of a knowledge graph, calculating the matching score according to the matching degree, then completing the estimated operation sequence required by the test instruction based on knowledge graph reasoning, counting the number of steps of the sequence, determining the step scores (the more the steps are, the higher the step scores are), identifying target pages (strange pages) which are not recorded in the estimated operation sequence by traversing all pages related to the estimated operation sequence, counting the number, determining the page scores (the more strange pages), the higher the page scores are accordingly, and finally weighting the similarity scores, the matching score, the step scores and the page scores according to preset weight coefficients, and finally obtaining quantized complexity parameters. The comprehensive evaluation of the multiple dimensions such as semantic similarity, knowledge graph matching degree, task operation difficulty, strange page risk and the like avoids the one-sided performance of single-dimension evaluation, and ensures the accuracy of complexity judgment. And the quantized complexity parameter can ensure the reliability of decision basis, and ensure the rationality and effectiveness of test strategy switching.

Specifically, the calculation complexity parameter can be quantitatively evaluated through multiple dimensions, the execution complexity of the test instruction can be evaluated by using the complexity parameter, and the execution difficulty of the test instruction can be quantitatively evaluated according to the size of the complexity parameter. For example, the dimensions such as the historical experience matching degree, the knowledge graph page matching degree, the estimated operation step number, the strange page number and the like of the test instruction can be determined by analyzing the test instruction, and the weighted calculation is performed according to the dimensions, so that the quantized complexity parameter is obtained. Under the condition that the complexity parameter is smaller than a first preset complexity threshold, a target operation sequence matched with the test instruction (for example, an operation sequence with highest cosine similarity is determined to be the target operation sequence through calculating cosine similarity) can be selected from an experience library, the path suitability of the target operation sequence is checked by combining the semantics and the knowledge graph (experience graph) of the current test instruction, the unmatched elements and paths are adaptively modified to generate a test operation sequence which can be directly executed, and under the condition that the complexity parameter is larger than the first preset complexity threshold, all executable paths of all test sub-instructions can be traversed based on the knowledge graph to generate a candidate path set in a summarizing mode, and according to the number of execution steps, stability and connection rationality, the optimal target paths between adjacent test sub-instructions are selected from the candidate path set, and finally all target paths are spliced in sequence to generate the complete test operation sequence.

It can be understood that the generation mode of the test operation sequence is adaptively selected according to the comparison of the complexity parameter and the first preset complex threshold value, and the effects of rapidly processing simple and repeated tasks through an experience library, coping with complex and strange scenes through a knowledge graph traversal path and remarkably improving the application range and robustness of the system are achieved while the test efficiency and the scene coverage capability are considered. Compared with a pure LLM reasoning scheme, the method of multiplexing the test operation sequence can be improved by more than a plurality of times in test speed, compared with a pure script scheme, various strange and dynamic change scenes can be processed, the number of API calls can be reduced by more than about half, and cloud service cost is effectively reduced.

In some embodiments, step 0113, selecting a target operation sequence matching the test instruction from the experience library if the complexity parameter is less than the first preset complexity threshold, comprises:

And 01131, determining the semantic vector with the highest similarity score as a target semantic vector, and generating a target operation sequence according to the operation step corresponding to the target semantic vector.

The target semantic vector may be an operation sequence semantic vector with the highest similarity score with the test semantic vector in the experience library.

The operation steps can be a directly executable GUI interaction action set stored in association with the semantic vector in the experience library, and the GUI interaction action set comprises information such as operation types, target elements, execution sequences and the like.

The target operation sequence may be a standardized operation flow which is obtained based on target semantic vector matching, is successfully executed in history and is matched with the current test instruction semantics.

Specifically, for example, the test cases in the existing framework are independent from each other, and similar operation sequences cannot be automatically identified and multiplexed. When executing two test tasks which are similar in nature, namely 'search cola' and 'search mineral water', a complete test flow needs to be written and executed respectively, and the existing general mode of 'search task' cannot be

utilized. In the embodiment of the application, under the condition that the complexity parameter is smaller than the first preset complexity threshold, the optimal operation sequence successfully executed by the history is positioned by taking the semantic similarity as a matching basis according to the high-dimensional vector quick retrieval capability of the vector database, a complex path reasoning link is skipped, and the initial generation of the test sequence is finished by directly multiplexing the history experience, so that the high-efficiency processing of a simple test task is realized. The semantic vector matching is utilized to realize the quick retrieval of similar tasks, the traditional keyword or coordinate matching is replaced, and the efficiency and accuracy of historical experience multiplexing are remarkably improved. The target sequence is directly generated based on the operation step corresponding to the semantic vector with the highest matching degree, so that the rapid execution of the conventional test task is realized, and the test period is greatly shortened.

Optionally, step 0113, modifying the target sequence of operations based on the test instructions and the empirical spectrum to generate a test sequence of operations, includes:

step 01132, carrying out semantic analysis on the test instruction to obtain a test page node, a test element node and a test conversion relation among the nodes corresponding to the test instruction;

Dividing the test page node, the test element node and the test conversion relation among the nodes corresponding to the test instruction according to the experience map to obtain effective parameters and failure parameters, wherein the effective parameters comprise the effective test page node, the effective test element node and the effective test conversion relation matched with the test conversion relation among the test page node, the test element node and the nodes corresponding to the test instruction in the experience map, and the failure parameters comprise the effective test page node, the effective test element node and the effective test conversion relation which are not matched with the test conversion relation among the test page node, the test element node and the nodes corresponding to the test instruction in the experience map;

Step 01134, selecting a replacement parameter matched with the failure parameter from the empirical spectrum according to a preset similarity condition;

And 01135, modifying the target operation sequence based on the effective parameter and the replacement parameter to generate a test operation sequence.

The test page node may be a GUI page type node (e.g., search page, item detail page, etc.) related to the obtained test task after the test instruction is semantically parsed.

The test element node may be a GUI interactive element node (such as a search box, a purchase button, etc.) operated by the obtained test task after the semantic analysis of the test instruction.

The test conversion relationship may be a logic association relationship of skip and element triggering operation among pages obtained after semantic analysis of the test instruction.

The effective parameters can be test page nodes, test element nodes, parameter sets which are completely matched with experience patterns and can be directly multiplexed in a test conversion relation, and the like.

The failure parameters may be a set of parameters that are not matched with the experience patterns and cannot be directly reused in the test page node, the test element node and the test conversion relation (for example, failure parameters caused by UI modification, element position or name change, etc.).

The preset similarity condition may be a preset semantic similarity threshold, which is used for screening the replacement parameters matched with the failure parameters from the experience map.

The replacement parameters can be page nodes, element nodes or conversion relations and the like which meet the preset similarity condition with the failure parameters in the experience map and can replace the execution of the failure parameters.

The modification of the target operation sequence may be a process of adaptively adjusting the target operation sequence obtained by matching the experience library based on the effective parameter and the replacement parameter, so as to adapt to the current GUI interface state.

Specifically, under the condition that the complexity parameter is smaller than a first preset complexity threshold, parameters such as test page nodes, test element nodes, inter-node test conversion relations and the like corresponding to the test tasks can be obtained through

multi-mode semantic analysis on the test instructions. Matching and matching the three parameters with the experience patterns one by one, wherein the parameters which are completely matched with the patterns are marked as effective parameters and can be directly multiplexed, and the parameters which are not matched with the patterns and cannot be directly executed are marked as failure parameters and need to be replaced. And then, searching page nodes, element nodes or conversion relations which are closest to the failure parameter semantics in the experience map according to the preset similarity condition, determining the page nodes, the element nodes or the conversion relations as replacement parameters, finally, reserving effective parameters in the target operation sequence, replacing the failure parameters with the replacement parameters, finishing the correction of the target operation sequence, and finally generating an executable test operation sequence which is adaptive to the current GUI interface.

It should be noted that the preset similarity condition may include at least one of semantic vector similarity and visual similarity, for example, a node (or parameter) having the highest semantic vector similarity with the failure parameter may be substituted, in the case where there is no semantic vector similarity, a node (or parameter) having the highest visual similarity may be used as a substitute, in the case where there is no visual similarity, a new node may be directly created or the failure parameter may be defined as skip logic. By identifying effective parameters and failure parameters and selecting replacement parameters from the experience atlas, scenes such as interface element positioning change, page structure fine adjustment and the like can be automatically adapted, and test effectiveness can be maintained without manually modifying scripts. The new relation between the failure parameters and the replacement parameters is updated to the system, so that continuous iteration and optimization of historical experience are realized, and the adaptability of the system to environmental changes is enhanced.

Optionally, in step 0114, in the case that the complexity parameter is greater than the first preset complexity threshold, traversing at least one execution path of each test sub-instruction based on the knowledge-graph, generating a candidate path set includes:

Step 01111, based on the knowledge graph, sequentially searching at least one execution path of each test sub-instruction;

step 01142, summarizing each execution path to generate a candidate path set.

The path summarizing may be a process of integrating and collecting the independent execution paths corresponding to the test sub-instructions according to the execution sequence of the test sub-instructions.

The method comprises the steps of carrying out a first test on a complex sub-instruction, carrying out a first test on the complex sub-instruction, carrying out a second test on the complex sub-instruction, carrying out a third test on the complex sub-instruction, and carrying out a first test on the complex sub-instruction, wherein the first test sub-instruction is a test sub-instruction, carrying out a second test sub-instruction, carrying out a first test on the complex sub-instruction, carrying out a second test sub-instruction, carrying out a third test on the complex sub-instruction, and carrying out a fourth test on the complex sub-instruction. For example, by sequentially traversing and searching all feasible execution paths corresponding to each test sub-instruction according to the sequence of the test sub-instructions after the test sub-instructions are disassembled based on the page nodes, element nodes and conversion relations of the knowledge graph, ensuring that each test sub-instruction obtains at least one executable path, and then uniformly integrating the execution paths of all the test sub-instructions according to the sequence of the test sub-instructions to form a candidate path set covering all the test sub-instructions. Through a graph traversal mode, all potential execution paths meeting the requirements of complex instructions can be found, test dead zones caused by fixed script logic are avoided, generation of a candidate path set completely depends on a constructed knowledge graph, and logic rigor and traceability of an reasoning process are guaranteed.

In some embodiments, step 0114, selecting a target path between adjacent test sub-instructions based on the candidate path set to generate a sequence of test operations includes:

Step 01143, sequentially scoring each execution path corresponding to the test sub-instruction according to the execution step number, the execution success rate and the strange parameter number included in the test sub-instruction, and determining the execution path corresponding to the test sub-instruction with the highest score as a target sub-execution path, wherein the strange parameter comprises a conversion relation among page nodes, element nodes or nodes which do not appear in the knowledge graph;

And 01144, sequentially connecting each target sub-execution path to generate a test operation sequence.

The number of execution steps may be the number of GUI operations required to complete the execution path corresponding to a single test sub-instruction, for measuring the path execution efficiency.

The execution success rate may be the successful execution probability of the execution path in the history test, and reflects the stability and reliability of the path.

The unfamiliar parameters may be unrecorded or unrecorded page nodes, element nodes or conversion relations among the nodes in the knowledge graph, and represent unknown risk degrees of the execution path.

The path scoring may be a single execution path comprehensive score obtained by weighting and calculating according to three indexes of the number of execution steps, the execution success rate and the number of strange parameters, and the single execution path comprehensive score is used for sorting the path quality.

The target sub-execution path may be the highest and optimal execution path of all execution paths corresponding to a single test sub-instruction.

The sequential connection may be a process of splicing the execution paths of the targets end to end and combining according to the original service execution sequence of the test sub-instruction.

The test operation sequence can be a complete GUI operation flow which is formed after the optimal sub-paths are spliced and is adaptive to high-complexity test tasks and can be directly and automatically executed.

Specifically, under the condition that the complexity parameter is larger than a first preset complex threshold, the overall optimal target sub-execution path can be screened according to the execution efficiency (step number), the running stability (success rate) and the unknown risk (strange parameter) serving as scoring basis, the quantization scoring of each sub-execution path is carried out, and then the optimal sub-paths are spliced in order according to the original business logic sequence of the test task to form an efficient, stable and low-risk complete test operation sequence, wherein the path rationality, the execution success rate and the flow continuity are considered in the complex scene.

The method comprises the steps of determining a target sub-execution path of each sub-instruction, determining an execution path with highest scoring under each sub-instruction, determining an original business execution sequence of the sub-instruction according to the test instruction, connecting all the target sub-execution paths end to end in sequence and integrating the target sub-execution paths into a complete flow according to an original business execution sequence of the sub-instruction, and finally generating a test operation sequence adapting to a high-complexity task. And comprehensively considering the execution efficiency, the historical success rate, the unknown risk and other factors to score the path, ensuring that the generated test operation sequence is the global optimal path, and improving the test success rate of the complex task.

In some embodiments, step 011, processing test instructions based on knowledge maps and/or experience libraries to generate a sequence of test operations, includes:

0115, under the condition that the complexity parameter is larger than or equal to a second preset complexity threshold value and smaller than or equal to a third preset complexity threshold value, acquiring a preliminary operation sequence through searching an experience library;

and 0116, verifying and modifying the preliminary operation sequence by using the knowledge graph to generate a test operation sequence.

The second preset complex threshold and the third preset complex threshold may be preset parameters, and the second preset complex threshold and the third preset complex threshold are matched to divide a task complexity interval, that is, under the condition that the complexity parameter is between the second preset complex threshold and the third preset complex threshold, the test task corresponding to the test instruction may be regarded as a medium-complexity task, which is a task type that cannot be stably executed only by pure experience multiplexing and does not need full-scale graph reasoning.

The preliminary operation sequence can be an initial operation flow which is obtained quickly through experience library semantic vector retrieval and is matched with the test instruction semanteme, and the initial operation flow is an intermediate result to be checked and corrected.

Specifically, aiming at the testing task of complexity parameters which are not suitable for pure experience multiplexing and do not need full-scale intelligent reasoning, the basic operation sequence can be rapidly acquired through an experience library to ensure the generation efficiency, then the knowledge graph is utilized to complete path validity verification and failure link restoration to ensure the execution reliability, and the dynamic balance of efficiency and accuracy is realized. For example, in the case that the complexity parameter of the test instruction is greater than or equal to the second preset complexity threshold and less than or equal to the third preset complexity threshold, the semantic similarity search may be performed in the experience library by converting the test instruction into a test semantic vector, and

the preliminary operation sequence closest to the test instruction semantic may be obtained by matching. And for the step of verifying failure, the preliminary operation sequence is corrected by selecting the replacement parameters of semantic matching from the knowledge graph, and the final test operation sequence which can be directly executed is generated after verification and modification. The time consumption of sequence generation is shortened through fast retrieval of an experience library, the failure of the execution is avoided through light-weight correction of the knowledge graph, and the processing speed and the success rate of the test instruction are balanced.

It should be noted that, taking the experience mode and the reasoning mode as examples, in the embodiment of the present application, when the execution of the acquisition multiplexing path in the experience mode fails, a new test policy may be generated by automatically switching to the reasoning path and starting the LLM deep analysis mode.

In certain embodiments, the method further comprises:

Step 013, acquiring the execution state of each target sub-execution path, wherein the execution state comprises execution success and execution failure;

Step 014, updating the experience library with the semantic feature vector corresponding to the target sub-execution path whose execution state is successful;

And 015, updating the knowledge graph according to the semantic analysis result corresponding to the target sub-execution path with the execution state being successful.

The execution state may be a final running result of the target sub-execution path in the GUI automation execution process, including execution success and execution failure.

The updating of the experience library can be a process of updating the semantic feature vector and the operation step corresponding to the target sub-execution path which is successfully executed to the vector database, and the settling of test experience is realized.

The semantic analysis result may be structural knowledge such as page type, element semantic, operation causal relationship, conversion relationship, and the like, which are obtained after multi-mode semantic analysis is performed on the target sub-execution path which is successfully executed.

The knowledge graph updating can be a process of supplementing a structural semantic analysis result corresponding to the target sub-execution path which is successfully executed to a graph database to perfect page nodes, element nodes and conversion relations.

Specifically, after the execution of the test operation sequence is completed, the client uploads the execution result to the server in real time, the server automatically collects the execution state, performs bidirectional knowledge precipitation on the effective path which is successfully executed, converts path semantics into vectors, stores the vectors into an experience library, supplements structured semantic knowledge to a knowledge graph, and enables the knowledge library of the test system to continuously and abundantly complete upgrading along with the increase of the execution times. After the test operation sequence is executed, the execution state of each sub-path can be confirmed to be successful or failed by collecting and judging the running results of each target sub-path one by one. Extracting service semantics and generating semantic feature vectors of target sub-execution paths with successful execution states, correlating the semantic feature vectors with corresponding operation steps, updating the semantic feature vectors to an experience library in an increment mode to serve as new reusable historical experiences, extracting multi-mode semantic analysis results (page nodes, element nodes, operation conversion relations and the like) of the target sub-execution paths with successful execution states, updating the structured knowledge increment to a knowledge graph (graph database), perfecting a GUI semantic network, and improving the self-updating characteristic of the system, wherein the system experiences are increasingly enriched along with the accumulation of using times, the intelligent testing level and the success rate are continuously improved, so that the workload of manually maintaining the knowledge graph and the experience library is greatly reduced, and the low cost and the high efficiency of long-term operation and maintenance are realized.

In certain embodiments, the method further comprises:

Step 016, under the condition that the execution failure occurs in the execution test operation sequence, performing multi-mode fault diagnosis, wherein the fault diagnosis comprises the steps of identifying the failure type and acquiring a repair strategy;

and step 017, updating the repaired test operation sequence to a knowledge graph and an experience library.

The execution failure can be an abnormal event that the test operation sequence fails in element positioning, page jump is abnormal, operation is overtime, popup window interception and the like and cannot complete a test instruction according to expectations in the GUI automatic execution process.

The multi-mode fault diagnosis can be a diagnosis process of fusing multidimensional data such as a visual mode (page screenshot), a text mode (operation log or instruction semantics), a behavior mode (operation sequence execution record) and the like, and analyzing and positioning the execution failure root cause through a multi-mode large model.

The failure type may be a classification result which is identified by multi-mode fault diagnosis and characterizes the nature of execution failure, including but not limited to core failure types such as element positioning failure, page state abnormality, operation logic conflict, permission verification interception and the like.

The repair policy may be a standardized solution preset for different failure types and capable of being automatically executed, including element re-identification, path re-planning, parameter dynamic replacement, popup avoidance processing, and the like.

For example, under the condition that the test cannot be performed due to element positioning failure, the multi-mode large model can be called to perform semantic analysis on the current page based on multi-mode operation track data such as a screenshot of a page, a control tree and the like when the execution fails, so that element semantics, visual characteristics and position information of interactable elements in the page are extracted, the elements are re-identified without relying on positioning information such as original fixed IDs, XPath and the like, then the semantic characteristics of the elements to be positioned are matched with an analysis result, the latest position and semantic information of the target elements are determined, and element positioning parameters in a test operation sequence are corrected based on the newly identified element information, so that an executable operation sequence after repair is generated;

For another example, under the condition that the page flow change cannot be tested, the difference between the current actual page flow and the expected flow can be compared based on the page nodes, element nodes and conversion relations stored in the knowledge graph, so that the flow failure node is positioned, then, according to the business logic of the test instruction, the original test instruction is disassembled again into test sub-instructions, the execution paths of the test sub-instructions are traversed based on the knowledge graph, a candidate path set is generated, then, the paths are scored according to the number of execution steps, the execution success rate and the number of strange parameters, so that a target sub-execution path is selected, and finally, each target sub-execution path is connected according to the business logic to replace the original failure flow, so that a test operation sequence adapting to the new page flow is generated;

For another example, under the condition that the interface is modified and cannot be tested, semantic analysis can be performed on the modified interface by calling the multi-mode large model, so that semantic analysis results such as page types, element semantics, operation causal relationships and the like of a new page are obtained, then attribute information of original nodes is corrected through newly added page nodes and element nodes after the modification, conversion relationships among the nodes are updated, failure nodes, conversion relationships and the like are deleted, so that a knowledge graph is updated based on the analysis results, and finally a test operation sequence adapting to the modified interface is regenerated by combining an experience base based on the updated knowledge graph;

It can be understood that after the test operation sequence is regenerated, the regenerated successfully executed test operation sequence can be subjected to semantic vectorization processing to generate a corresponding semantic feature vector, the semantic vector and the repaired operation step increment are stored in an experience base to replace or mark the original failure operation sequence so as to update the experience base, meanwhile, the semantic analysis result corresponding to the repaired successfully executed operation sequence, including the newly added or corrected page nodes, element nodes and conversion relations among the nodes, can be updated to a knowledge graph in an increment mode, the GUI semantic network is perfected, and after the updating, the repaired operation sequence and knowledge can be directly reused when the similar test instructions are processed later, and repeated diagnosis and repair are not needed.

The multi-modal large model can be a large model (such as GPT-4V, claude-3.5) integrating visual understanding and semantic resolving capability, and is a computing carrier for realizing multi-modal fault diagnosis and root cause positioning.

The repaired test operation sequence can be an effective operation flow which is obtained by performing self-adaptive correction and optimization on the original test operation sequence and can be re-executed after the repair strategy is obtained through fault diagnosis.

The method comprises the steps of discarding an inefficient mode of traditional manual investigation and repair when a test operation sequence fails to execute, merging multi-dimensional data such as screenshot, logs and operation sequences of a failure site according to the cross-modal analysis capability of a multi-modal large model, accurately identifying failure types and root causes, matching a corresponding repair strategy to automatically correct the operation sequence to complete self-healing of faults, synchronously updating the repaired effective sequence to a knowledge graph and an experience library, precipitating repair experience to be system knowledge, forming a self-learning closed loop from execution failure to diagnosis repair, knowledge update and follow-up multiplexing, and enabling the test system to have core characteristics of dynamically adapting to abnormal execution and continuously optimizing the self-healing capability.

For example, capturing current page screenshot (visual mode), test operation sequence execution log (text mode), test instruction original semantics (text mode), operation context (behavior mode) of failed nodes and the like when the execution fails to acquire multi-mode data in the failure state, then calling a multi-mode large model to analyze the acquired multi-mode data, for example, identifying visual anomalies such as interface element states, popup shielding, layout changes and the like by combining the page screenshot, or executing semantic anomalies such as logic conflict, parameter mismatch and the like by combining the operation log and instruction semantic positioning, or comprehensively judging specific types (such as element positioning failure, page jump failure, authority interception anomalies and the like) of the execution failure, then matching corresponding standardized solutions from a repair strategy base preset by a system according to the identified failure types, definitely repairing directions (such as element positioning failure calls visual model re-identification elements, page jump anomalies are based on knowledge re-planning paths), finally carrying out self-adaption correction, parameter replacement or logic re-planning on the test operation sequence of the original execution failure, generating a re-executable post-repair test operation sequence, extracting service vectors of the post-repair operation sequence, generating feature vectors of the post-repair operation sequence, updating the feature vectors to the corresponding feature vectors to a corresponding operation pattern, updating the feature vectors to a corresponding operation map, updating the feature vectors to a corresponding rule-updating relation to a test operation map, updating the corresponding relation to a test node, a new rule-updating relation, a corresponding relation to a test node, a new rule, and a new rule-updating rule, and a new rule, and a corresponding relation to a new rule, the integrity and the accuracy of the GUI semantic network are perfected, the automatic self-healing of test execution is realized, the fault location, type identification and repair execution can be completed without manual intervention, the manual investigation and script modification cost after test failure is greatly reduced, the continuous and stable operation of a test flow is ensured, the effective operation sequence after repair is synchronously updated to a knowledge graph and an experience library, the fault repair experience is continuously precipitated, and the system can directly reuse similar repair schemes, so that the test intellectualization and self-healing capacity is continuously improved.

The automatic test method of the embodiment of the application can enable a test system to receive a test instruction in a natural language form (such as searching iPhone and checking details of a first commodity), automatically understand service semantics and operation intentions behind the instruction and convert the test instruction into an executable GUI operation sequence, enable the test system to automatically infer needed intermediate operation steps according to a high-level test target, intelligently select an optimal scheme in a plurality of feasible paths, process a dynamic page flow and a conditional popup window, convert GUI interaction data in historical test execution into a reusable knowledge asset, automatically identify and match similar test tasks, directly multiplex verified successful paths, automatically analyze root causes (such as element positioning failure and interface change) when the test instruction fails, execute a repair strategy for different failure types, learn from failure, dynamically update a knowledge base, finally fuse visual features (layout, color and icon) and text information of a screenshot, establish cross-modal association between visual feature-element function-operation intentions, process a Canvas, and other non-GUI rendering scenes.

Referring to fig. 3-5, the automatic test mode according to the embodiments of the present application will be further described under the structure of the server and the client.

The method comprises the steps of firstly collecting and uploading multimodal triplet data through a client in an offline stage construction stage, receiving instructions issued by a server by the client, executing specific GUI interaction on tested equipment, returning an execution result and real-time page data to the server, for example, obtaining an execution instruction containing an operation step sequence (clicking, inputting and the like), target element positioning information (XPath, semantic characteristics and coordinates) and an execution mode identifier from a server application service layer, sequentially executing GUI operations according to a step sequence issued by the server, wherein the execution instruction comprises an appointed action (such as clicking an element corresponding to XPath), collecting current page screenshot and control tree data in real time after the operation is completed, returning the current page screenshot and the control tree data to the server, recording a local execution log (step number, operation type, execution time and success or not) by the client, immediately stopping the current step when the client collects the current step, capturing a screenshot, a control tree and an error log when the fault, reporting the server in real time, and returning the final screenshot (page control tree data and service control tree) when all steps are not abnormally executed.

The client can also receive a server repair strategy instruction, execute repair operation and return a result, if the repair is successful, continue to execute the subsequent unfinished test steps, return the repaired operation sequence, the repaired execution data and the final page state to the server, if the repair instruction still fails after being executed, collect secondary fault data, report the secondary fault data to the server, stop the execution of the client and wait for manual operation, and if the fault repair is completed, the client supplements and uploads the complete operation log and the triple data after the repair to the service server.

Taking the server layout multi-mode large model to realize the test operation sequence as an example for explanation. The server can receive the triple data uploaded by the client through an application service layer, after the checksum format is standardized, the triple data are forwarded TO an AI reasoning layer for semantic analysis, meanwhile, the original data are stored in a MongoDB for standby, the AI reasoning layer calls VLM/LLM TO conduct Page semantic analysis (identifying Page types, core functions, layout structures and key service elements) on the triple, element semantic analysis (extracting visual features of interface elements, interaction modes and service semantics), operation cause and effect analysis (judging target elements of operations, page change results and service intention), finally the analysis results are packaged into structural data of Page nodes, element nodes and transformation edges, the structured data generated by the AI reasoning layer are written into a Neo4j graph database, page/Element nodes are created according TO preset Schema, three types of relations of ' Page containing elements (CONTAINS) ' operation conversion (TRANSITION) ' Element semantic similarity (SIMILAR_SET) are established, finally, the operation sequence after the analysis is converted into a high-dimensional operation sequence is converted into a structural data of a Page node, the operation rule is set, the SIMILAR vectors of the operation rule is set, the SIMILAR vectors are set up, the SIMILAR vectors are created, the SIMILAR vectors are written into the database of the operation rules, and the SIMILAR vectors are created, and the SIMILAR vectors are written into the database.

The application service layer of the server can also receive a natural language test instruction, a target device ID and a tested application packet name, and perform format check on the instruction to generate a unique task ID, add the task into a Redis/Celery task queue, schedule according to the device load, and return a task queuing state to a tester. The application service layer combines the retrieval result of the knowledge storage layer and the analysis of the AI reasoning layer, calculates complexity parameters (the lower the score is, the simpler the task is, the more experience multiplexing is possible), for example, the test instruction is vectorized by calling the AI reasoning layer, the most similar historical operation sequence is retrieved in the Milvus vector database, the cosine similarity score is obtained, the AI reasoning layer is used for multi-modal identification, the initial page type of the historical sequence and the current page type of the client are compared, then LLM is called for carrying out task disassembly on the natural language instruction, the number of operation steps required by the test is estimated and normalized, the page type related in the LLM is called again, whether the query exists in the Neo4j graph database or not is judged, the more strange page number is required, the higher the penalty score is, the application service layer selects the corresponding test operation sequence generation mode according to the complexity parameters, the operation instruction is issued to the client after the test operation sequence is generated, the application service layer receives the execution feedback (each operation result, page screenshot and control tree) of the client in real time, the execution log is recorded, if the execution failure is detected (if the element is not found, the page is not triggered immediately, and the task is suspended). After the client finishes the test, the server automatically verifies the test result and judges whether the test result accords with the expectation of the natural language instruction. The method comprises the steps of calling LLM at an AI reasoning layer to extract expected results of test instructions, calling VLM to combine a final page screenshot and a control tree returned by a client to perform multi-mode verification, judging success or failure of the test and giving a verification reason, storing all information (instructions, execution modes, steps, results and logs) of a test task into MongoDB by an application service layer to form a test record, converting a current operation sequence into a semantic vector by a knowledge storage layer if the test is successfully executed, adding the semantic vector into a Milvus vector database, realizing automatic increment precipitation of historical experience, and marking failure reasons and triggering a self-healing optimization flow if the test is failed.

In order to facilitate better implementation of the automatic test method of the embodiment of the application, the embodiment of the application also provides an automatic test device. Referring to the drawings, the structure of an automatic test device according to an embodiment of the application is shown. The automatic test equipment 200 may include:

the generation module 201 is configured to process a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, where the knowledge graph is constructed based on a result obtained by performing semantic analysis on data of a test operation, and the knowledge graph at least includes page nodes, element nodes and conversion relations between the nodes;

an execution module 202, configured to execute the test operation sequence.

The various elements of the automatic test equipment described above may be implemented in whole or in part by software, hardware, and combinations thereof. The above units may be embedded in hardware or may be independent of a processor in the computer device, or may be stored in software in a memory in the computer device, so that the processor invokes and executes operations corresponding to the above units.

The automatic test equipment 200 may be integrated in a terminal or a server having a memory and a processor mounted therein and having an arithmetic capability, or the automatic test equipment 200 may be the terminal or the server.

Optionally, the present application further provides a computer device, including a memory and a processor, where the memory stores a computer program, and the processor implements the steps in the above method embodiments when executing the computer program.

The diagram is a schematic structural diagram of a computer device provided in an embodiment of the present application, where the computer device may be a terminal or a server. As shown, the computer device 300 includes a processor 301 having one or more processing cores, a memory 302 having one or more computer readable storage media, and a computer program stored on the memory 302 and executable on the processor. The processor 301 is electrically connected to the memory 302. It will be appreciated by those skilled in the art that the computer device structure shown in the figures is not limiting of the computer device and may include more or fewer components than shown, or may combine certain components, or a different arrangement of components.

The processor 301 is a control center of the computer device 300, connects various parts of the entire computer device 300 using various interfaces and lines, and performs various functions of the computer device 300 and processes data by running or loading software programs and/or modules stored in the memory 302, and calling data stored in the memory 302, thereby performing overall processing of the computer device 300.

In the embodiment of the present application, the processor 301 in the computer device 300 loads the instructions corresponding to the processes of one or more computer programs into the memory 302 according to the following steps, and the processor 301 executes the computer programs stored in the memory 302, so as to implement various functions:

the method comprises the steps of processing a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and at least comprises page nodes, element nodes and conversion relations among the nodes;

And executing the test operation sequence.

The specific implementation of each operation above may be referred to the previous embodiments, and will not be described herein.

Optionally, as shown, the computer device 300 further comprises a display 303, radio frequency circuitry 304, audio circuitry 305, an input unit 306, and a power supply 307. The processor 301 is electrically connected to the display 303, the radio frequency circuit 304, the audio circuit 305, the input unit 306, and the power supply 307, respectively. It will be appreciated by those skilled in the art that the computer device structure shown in the figures is not limiting of the computer device and may include more or fewer components than shown, or may combine certain components, or a different arrangement of components.

The display 303 may be used to display a graphical user interface and receive operation instructions generated by a user acting on the graphical user interface. The display 303 may include a display panel and a touch panel. Wherein the display panel may be used to display information entered by a user or provided to a user as well as various graphical user interfaces of a computer device, which may be composed of graphics, text, icons, video, and any combination thereof. The touch panel may be used to collect touch operations on or near the user (such as operations on or near the touch panel by the user using any suitable object or accessory such as a finger, stylus, etc.), and generate corresponding operation instructions, and the operation instructions execute corresponding programs. Alternatively, the touch panel may include two parts, a touch detection device and a touch controller. The touch controller receives touch information from the touch detection device, converts the touch information into touch point coordinates, sends the touch point coordinates to the processor 301, and can receive and execute commands sent by the processor 301. The touch panel may overlay the display panel, and upon detection of a touch operation thereon or thereabout, the touch panel is passed to the processor 301 to determine the type of touch event, and the processor 301 then provides a corresponding visual output on the display panel in accordance with the type of touch event. In the embodiment of the present application, the touch panel and the display panel may be integrated into the display screen 303 to realize the input and output functions. In some embodiments, however, the touch panel and the touch panel may be implemented as two separate components to perform the input and output functions. I.e. the display 303 may also implement an input function as part of the input unit 306.

The radio frequency circuitry 304 may be used to transceive radio frequency signals to establish wireless communications with a network device or other computer device via wireless communications.

The audio circuit 305 may be used to provide an audio interface between a user and a computer device through a speaker, microphone. The audio circuit 305 may convert the received audio data into an electrical signal, transmit the electrical signal to a speaker, and convert the electrical signal to a sound signal for output by the speaker, and on the other hand, the microphone converts the collected sound signal

into an electrical signal, receives the electrical signal from the audio circuit 305, converts the electrical signal into audio data, processes the audio data with the output processor 301, sends the audio data to another computer device, for example, via the radio frequency circuit 304, or outputs the audio data to the memory 302 for further processing. The audio circuit 305 may also include an ear bud jack to provide communication of the peripheral ear bud with the computer device.

The input unit 306 may be used to receive input numbers, character information, or object feature information (e.g., fingerprint, iris, facial information, etc.), and to generate keyboard, mouse, joystick, optical, or trackball signal inputs related to user settings and function control.

The power supply 307 is used to power the various components of the computer device 300. Alternatively, the power supply 307 may be logically connected to the processor 301 through a power management system, so as to perform functions of managing charging, discharging, and power consumption management through the power management system. The power supply 307 may also include one or more of any components, such as a direct current or alternating current power supply, a recharging system, a power failure detection circuit, a power converter or inverter, a power status indicator, and the like.

Although not shown in the figures, the computer device 300 may further include a camera, a sensor, a wireless fidelity module, a bluetooth module, etc., which will not be described herein.

The present application also provides a computer-readable storage medium storing a computer program. The computer readable storage medium may be applied to a computer device, and the computer program causes the computer device to execute the corresponding flow in the automatic test method in the embodiment of the present application, which is not described herein for brevity.

The present application also provides a computer program product comprising computer instructions stored in a computer readable storage medium. The processor of the computer device reads the computer instructions from the computer readable storage medium, and the processor executes the computer instructions, so that the computer device executes the corresponding flow in the automatic test method in the embodiment of the present application, which is not described herein for brevity.

The present application also provides a computer program comprising computer instructions stored in a computer readable storage medium. The processor of the computer device reads the computer instructions from the computer readable storage medium, and the processor executes the computer instructions, so that the computer device executes the corresponding flow in the automatic test method of the present application, which is not described herein for brevity.

It should be appreciated that the processor of the present application may be an integrated circuit chip having signal processing capabilities. In implementation, the steps of the above method embodiments may be implemented by integrated logic circuits of hardware in a processor or instructions in software form. The Processor may be a general purpose Processor, a digital signal Processor (DIGITAL SIGNAL Processor, DSP), an Application SPECIFIC INTEGRATED Circuit (ASIC), an off-the-shelf programmable gate array (Field Programmable GATE ARRAY, FPGA) or other programmable logic device, discrete gate or transistor logic device, discrete hardware components. The disclosed methods, steps, and logic blocks in the embodiments of the present application may be implemented or performed. A general purpose processor may be a microprocessor or the processor may be any conventional processor or the like. The steps of the method disclosed in connection with the embodiments of the present application may be embodied directly in the execution of a hardware decoding processor, or in the execution of a combination of hardware and software modules in a decoding processor. The software modules may be located in a random access memory, flash memory, read only memory, programmable read only memory, or electrically erasable programmable memory, registers, etc. as well known in the art. The storage medium is located in a memory, and the processor reads the information in the memory and, in combination with its hardware, performs the steps of the above method.

It will be appreciated that the memory in embodiments of the application may be volatile memory or nonvolatile memory, or may include both volatile and nonvolatile memory. The nonvolatile Memory may be a Read-Only Memory (ROM), a Programmable ROM (PROM), an Erasable PROM (EPROM), an Electrically Erasable EPROM (EEPROM), or a flash Memory. The volatile memory may be random access memory (Random Access Memory, RAM) which acts as external cache memory. By way of example, and not limitation, many forms of RAM are available, such as static random access memory (STATIC RAM, SRAM), dynamic random access memory (DYNAMIC RAM, DRAM), synchronous Dynamic Random Access Memory (SDRAM), double data rate Synchronous dynamic random access memory (Double DATA RATE SDRAM, DDR SDRAM), enhanced Synchronous dynamic random access memory (ENHANCED SDRAM, ESDRAM), synchronous link dynamic random access memory (SYNCHLINK DRAM, SLDRAM), and Direct memory bus RAM (DR RAM). It should be noted that the memory of the systems and methods described herein is intended to comprise, without being limited to, these and any other suitable types of memory.

Those of ordinary skill in the art will appreciate that the various illustrative elements and algorithm steps described in connection with the embodiments disclosed herein may be implemented as electronic hardware, or combinations of computer software and electronic hardware.

Whether such functionality is implemented as hardware or software depends upon the particular application and design constraints imposed on the solution. Skilled artisans may implement the described functionality in varying ways for each particular application, but such implementation decisions should not be interpreted as causing a departure from the scope of the present application.

It will be clear to those skilled in the art that, for convenience and brevity of description, specific working procedures of the above-described systems, apparatuses and units may refer to corresponding procedures in the foregoing method embodiments, and are not repeated herein.

In the present embodiment, the term "module" or "unit" refers to a computer program or a part of a computer program having a predetermined function and working together with other relevant parts to achieve a predetermined object, and may be implemented in whole or in part by using software, hardware (such as a processing circuit or a memory), or a combination thereof. Also, a processor (or multiple processors or memories) may be used to implement one or more modules or units. Furthermore, each module or unit may be part of an overall module or unit that incorporates the functionality of the module or unit.

In the several embodiments provided by the present application, it should be understood that the disclosed systems, devices, and methods may be implemented in other manners. For example, the apparatus embodiments described above are merely illustrative, e.g., the division of the units is merely a logical function division, and there may be additional divisions when actually implemented, e.g., multiple units or components may be combined or integrated into another system, or some features may be omitted or not performed. Alternatively, the coupling or direct coupling or communication connection shown or discussed with each other may be an indirect coupling or communication connection via some interfaces, devices or units, which may be in electrical, mechanical or other form.

The units described as separate units may or may not be physically separate, and units shown as units may or may not be physical units, may be located in one place, or may be distributed on a plurality of network units. Some or all of the units may be selected according to actual needs to achieve the purpose of the solution of this embodiment.

In addition, each functional unit in the present application may be integrated in one processing unit, each unit may exist alone physically, or two or more units may be integrated in one unit.

The functions, if implemented in the form of software functional units and sold or used as a stand-alone product, may be stored in a computer-readable storage medium. Based on this understanding, the technical solution of the present application may be embodied essentially or in a part contributing to the prior art or in the form of a software product stored in a storage medium, comprising several instructions for causing a computer device (which may be a personal computer, a server) to perform all or part of the steps of the method according to the embodiments of the present application. The storage medium includes various media capable of storing program codes such as a U disk, a mobile hard disk, a ROM, a RAM, a magnetic disk or an optical disk.

The foregoing is merely illustrative of the present application, and the present application is not limited thereto, and any person skilled in the art will readily recognize that variations or substitutions are within the scope of the present application. Therefore, the protection scope of the present application shall be subject to the protection scope of the claims.

Document Claims

Language: (English)

1. An automatic test method, comprising:

the method comprises the steps of processing a test instruction based on a knowledge graph and/or an experience library to generate a test operation sequence, wherein the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and at least comprises page nodes, element nodes and conversion relations among the nodes;

And executing the test operation sequence.

2. The automated testing method of claim 1, wherein the processing the test instructions based on the knowledge-graph and/or experience library to generate a sequence of test operations comprises:

And calling the multi-mode large model, and carrying out semantic understanding on the test instruction by combining a knowledge graph and/or an experience library so as to generate a test operation sequence.

3. The automatic test method according to claim 2, wherein the multi-modal large model is trained by multi-modal operation trajectory data, and the multi-modal operation trajectory data includes at least first page data before execution of the test operation, second page data after execution of the test operation, and test operation action data.

4. The automated test method of claim 2, further comprising:

the multi-mode large model performs semantic analysis on the multi-mode operation track data to obtain semantic analysis results, wherein the semantic analysis results comprise page types before and after test operation execution, page element semantics and operation causal relationships, and operation sequences obtained through analysis are subjected to semantic vectorization and stored in the experience library.

5. The automated testing method of claim 1, wherein the processing the test instructions based on the knowledge-graph and/or experience library to generate a sequence of test operations comprises:

Performing complexity evaluation on the test instruction to obtain a complexity parameter;

Under the condition that the complexity parameter is smaller than a first preset complexity threshold value, selecting a target operation sequence matched with the test instruction from the experience library, and modifying the target operation sequence based on the test instruction and the experience atlas to generate the test operation sequence;

And under the condition that the complexity parameter is larger than a first preset complexity threshold, traversing at least one execution path of each test sub-instruction based on the knowledge graph, generating a candidate path set, and selecting a target path between adjacent test sub-instructions based on the candidate path set to generate the test operation sequence, wherein the test sub-instructions are obtained after the test instructions are disassembled.

6. The automatic test method of claim 5 wherein said evaluating the complexity of the test instruction to obtain a complexity parameter comprises:

performing similarity calculation on a test semantic vector and semantic vectors of each operation sequence stored in the experience library to obtain each similarity score, determining the similarity score with the largest numerical value as a similarity score, and performing semantic vectorization on the test semantic vector based on the test instruction to obtain the test semantic vector;

carrying out semantic analysis on the test instruction to obtain a test semantic analysis result, carrying out matching calculation on the test semantic analysis result and the knowledge graph, and determining a matching degree score according to the calculation result;

Calculating an estimated operation sequence for executing the test instruction based on the knowledge graph, and determining step scores according to the number of steps contained in the estimated operation sequence;

Calculating the number of target pages appearing in the estimated operation sequence, and determining page scores, wherein the target pages comprise pages which are not recorded in the knowledge graph;

And carrying out weight calculation on the similarity score, the matching score, the step score and the page score to determine a complexity parameter.

7. The automatic test method of claim 6 wherein, in the event that the complexity parameter is less than a first preset complexity threshold, selecting a target sequence of operations from the library that matches the test instruction comprises:

and determining the semantic vector with the highest similarity score as a target semantic vector, and generating a target operation sequence according to the operation steps corresponding to the target semantic vector.

8. The automatic test method of claim 6 or 7, wherein modifying the target sequence of operations based on the test instructions and the empirical spectrum to generate the sequence of test operations comprises:

Carrying out semantic analysis on the test instruction to obtain a test page node, a test element node and a test conversion relation among nodes corresponding to the test instruction;

Dividing test page nodes, test element nodes and test conversion relations among the nodes corresponding to the test instructions according to the experience patterns to obtain effective parameters and failure parameters, wherein the effective parameters comprise effective test page nodes, effective test element nodes and effective test conversion relations matched with the test conversion relations among the test page nodes, the test element nodes and the nodes corresponding to the test instructions in the experience patterns, and the failure parameters comprise effective test page nodes, effective test element nodes and effective test conversion relations which are not matched with the test conversion relations among the test page nodes, the test element nodes and the nodes corresponding to the test instructions in the experience patterns;

Selecting a replacement parameter matched with the failure parameter from the empirical spectrum according to a preset similarity condition;

And modifying the target operation sequence based on the effective parameter and the replacement parameter to generate the test operation sequence.

9. The automatic test method of claim 5 wherein, in the event that the complexity parameter is greater than a first preset complexity threshold, traversing at least one execution path of each test sub-instruction based on the knowledge-graph, generating a set of candidate paths comprises:

Based on the knowledge graph, sequentially searching at least one execution path of each test sub-instruction;

and summarizing the execution paths to generate the candidate path set.

10. The automatic test method of claim 9 wherein the selecting a target path between adjacent ones of the test sub-instructions based on the candidate path set to generate the sequence of test operations comprises:

Sequentially scoring each execution path corresponding to the test sub-instruction according to the execution step number, the execution success rate and the strange parameter number included in the test sub-instruction, determining the execution path corresponding to the test sub-instruction with the highest score as a target sub-execution path, wherein the strange parameter includes a conversion relation among page nodes, element nodes or nodes which do not appear in the knowledge graph;

And sequentially connecting each target sub-execution path to generate the test operation sequence.

11. The automated testing method of claim 5, wherein the processing the test instructions based on the knowledge-graph and/or experience library to generate a sequence of test operations further comprises:

under the condition that the complexity parameter is larger than or equal to a second preset complexity threshold value and smaller than or equal to a third preset complexity threshold value, acquiring a preliminary operation sequence through searching of the experience library;

and verifying and modifying the preliminary operation sequence by using the knowledge graph to generate the test operation sequence.

12. The automatic test method of claim 10 or 11, wherein the method further comprises:

Acquiring the execution state of each target sub-execution path, wherein the execution state comprises execution success and execution failure;

Updating the experience library by taking the execution state as a semantic feature vector corresponding to the target sub-execution path which is successfully executed;

and updating the knowledge graph by taking the execution state as a semantic analysis result corresponding to the target sub-execution path which is successfully executed.

13. The automated test method of claim 12, further comprising:

under the condition that the execution failure occurs in the execution of the test operation sequence, carrying out multi-mode fault diagnosis, wherein the fault diagnosis comprises the steps of identifying a failure type and acquiring a repair strategy;

and updating the repaired test operation sequence to a knowledge graph and an experience library.

14. An automatic test equipment, said equipment comprising:

The system comprises a generation module, a test operation sequence generation module and a test operation sequence generation module, wherein the generation module is used for processing a test instruction based on a knowledge graph and/or an experience library, the knowledge graph is constructed based on a result obtained by carrying out semantic analysis on data of a test operation, and the knowledge graph at least comprises page nodes, element nodes and conversion relations among the nodes;

and the execution module is used for executing the test operation sequence.

15. A computer readable storage medium, characterized in that the computer readable storage medium stores a computer program, which is adapted to be loaded by a processor for performing the automatic test method according to any of claims 1-13.

16. A computer device, characterized in that it comprises a processor and a memory, in which a computer program is stored, the processor being arranged to execute the automatic test method according to any of claims 1-13 by calling the computer program stored in the memory.

17. A computer program product comprising computer instructions which, when executed by a processor, implement the automatic test method of any one of claims 1-13.

Language: (Chinese)

1.一种自动测试方法，其特征在于，包括：

基于知识图谱和/或经验库处理测试指令以生成测试操作序列；所述知识图谱基于执行测试操作的数据进行语义解析后得到的结果构建，所述知识图谱至少包括页面节点、元素节点以及节点之间的转换关系；所述经验库用于存储操作序列的语义向量及对应的操作步骤；

执行所述测试操作序列。

2.根据权利要求1所述的自动测试方法，其特征在于，所述基于知识图谱和/或经验库处理测试指令以生成测试操作序列，包括：

调用多模态大模型，结合知识图谱和/或经验库对所述测试指令进行语义理解，以生成测试操作序列。

3.根据权利要求2所述的自动测试方法，其特征在于，所述多模态大模型采用多模态操作轨迹数据训练得到，所述多模态操作轨迹数据至少包括测试操作执行前的第一页面数据、测试操作执行后的第二页面数据和测试操作动作数据。

4.根据权利要求2所述的自动测试方法，其特征在于，所述方法还包括：

所述多模态大模型对所述多模态操作轨迹数据进行语义解析以得到语义解析结果，所述语义解析结果包括测试操作执行前和执行后的页面类型、页面的元素语义及操作因果关系；将解析得到的操作序列进行语义向量化，并存入所述经验库。

5.根据权利要求1所述的自动测试方法，其特征在于，所述基于知识图谱和/或经验库处理测试指令以生成测试操作序列，包括：

对所述测试指令进行复杂度评估，以得到复杂度参数；

在所述复杂度参数小于第一预设复杂阈值的情况下，从所述经验库中选取和所述测试指令匹配的目标操作序列，并基于所述测试指令和所述经验图谱，对所述目标操作序列进行修改，以生成所述测试操作序列；

在所述复杂度参数大于第一预设复杂阈值的情况下，基于所述知识图谱遍历各个测试子指令的至少一个执行路径，生成候选路径集，并基于所述候选路径集选取相邻的所述测试子指令之间的目标路径，以生成所述测试操作序列，其中，所述测试子指令基于所述测试指令拆解后得到。

6.根据权利要求5所述的自动测试方法，其特征在于，所述对所述测试指令进行复杂度评估，以得到复杂度参数，包括：

将测试语义向量和所述经验库中存储的各个操作序列的语义向量进行相似度计算，以得到各个相似度分数，并将数值最大的相似度分数确定为相似度得分，所述测试语义向量基于所述测试指令进行语义向量化得到；

对所述测试指令进行语义解析，以得到测试语义解析结果，并将所述测试语义解析结果和所述知识图谱进行匹配计算，根据计算结果确定匹配度得分；

基于所述知识图谱，计算执行所述测试指令的预估操作序列，根据所述预估操作序列包含的步骤数量，确定步骤得分；

计算所述预估操作序列中出现的目标页面数量，确定页面得分，所述目标页面包括所述知识图谱中未记录的页面；

对所述相似度得分、所述匹配度得分、所述步骤得分和所述页面得分进行权重计算，以确定复杂度参数。

7.根据权利要求6所述的自动测试方法，其特征在于，所述在所述复杂度参数小于第一预设复杂阈值的情况下，从所述经验库中选取和所述测试指令匹配的目标操作序列，包括：

将相似度分数最高的语义向量确定为目标语义向量，并根据所述目标语义向量对应的操作步骤生成目标操作序列。

8.根据权利要求6或7所述的自动测试方法，其特征在于，所述基于所述测试指令和所述经验图谱，对所述目标操作序列进行修改，以生成所述测试操作序列，包括：

对所述测试指令进行语义解析，得到所述测试指令对应的测试页面节点、测试元素节点以及节点之间的测试转换关系；

根据所述经验图谱，对所述测试指令对应的测试页面节点、测试元素节点以及节点之间的测试转换关系进行划分，以得到有效参数和失效参数，所述有效参数包括在所述经验图谱中和所述测试指令对应的测试页面节点、测试元素节点以及节点之间的测试转换关系匹配的有效测试页面节点、有效测试元素节点以及有效测试转换关系，所述失效参数包括在所述经验图谱中和所述测试指令对应的测试页面节点、测试元素节点以及节点之间的测试转换关系不匹配的有效测试页面节点、有效测试元素节点以及有效测试转换关系；

根据预设的相似度条件，从所述经验图谱中选取和所述失效参数匹配的替换参数；

基于所述有效参数和所述替换参数，对所述目标操作序列进行修改，以生成所述测试操作序列。

9.根据权利要求5所述的自动测试方法，其特征在于，所述在所述复杂度参数大于第一预设复杂阈值的情况下，基于所述知识图谱遍历各个测试子指令的至少一个执行路径，生成候选路径集，包括：

基于所述知识图谱，依次查找各个所述测试子指令的至少一个执行路径；

将各个所述执行路径进行汇总，以生成所述候选路径集。

10.根据权利要求9所述的自动测试方法，其特征在于，所述基于所述候选路径集选取相邻的所述测试子指令之间的目标路径，以生成所述测试操作序列，包括：

根据所述测试子指令的执行步骤数、执行成功率和所述测试子指令包括的陌生参数数量，依次为所述测试子指令对应的各个所述执行路径打分，并将打分最高的所述测试子指令对应的执行路径确定为目标子执行路径，所述陌生参数包括在所述知识图谱中未出现的页面节点、元素节点或节点之间的转换关系；

依次连接各个所述目标子执行路径，以生成所述测试操作序列。

11.根据权利要求5所述的自动测试方法，其特征在于，所述基于知识图谱和/或经验库处理测试指令以生成测试操作序列，还包括：

在所述复杂度参数大于或者等于第二预设复杂阈值且小于或者等于第三预设复杂阈值的情况下，通过所述经验库检索获取初步操作序列；

利用所述知识图谱对所述初步操作序列进行验证和修改，以生成所述测试操作序列。

12.根据权利要求10或11所述的自动测试方法，其特征在于，所述方法还包括：

获取各个所述目标子执行路径的执行状态，所述执行状态包括执行成功和执行失败；

将所述执行状态为执行成功的所述目标子执行路径所对应的语义特征向量，更新所述经验库；

将所述执行状态为执行失败的所述目标子执行路径所对应的语义解析结果，更新所述知识图谱。

13.根据权利要求12所述的自动测试方法，其特征在于，还包括：

在执行所述测试操作序列出现执行失败的情况下，进行多模态故障诊断，所述故障诊断包括识别失败类型和获取修复策略；

将修复后的测试操作序列更新至知识图谱和经验库。

14. 一种自动测试装置，其特征在于，所述装置包括：

生成模块，用于基于知识图谱和/或经验库处理测试指令以生成测试操作序列；所述知识图谱基于执行测试操作的数据进行语义解析后得到的结果构建，所述知识图谱至少包括页面节点、元素节点以及节点之间的转换关系；所述经验库用于存储操作序列的语义向量及对应的操作步骤；

执行模块，用于执行所述测试操作序列。

15. 一种计算机可读存储介质，其特征在于，所述计算机可读存储介质存储有计算机程序，所述计算机程序适于处理器进行加载，以执行如权利要求1-13任一项所述的自动测试方法。

16. 一种计算机设备，其特征在于，所述计算机设备包括处理器和存储器，所述存储器中存储有计算机程序，所述处理器通过调用所述存储器中存储的所述计算机程序，用于执行权利要求1-13任一项所述的自动测试方法。

17. 一种计算机程序产品，包括计算机指令，其特征在于，所述计算机指令被处理器执行时实现权利要求1-13任一项所述的自动测试方法。
